Arya T S

M.Tech Artificial Intelligence & Software Engineering
Cochin University of Science and Technology (CUSAT)

October 2026

Project Overview

Part I – Foundations

- N-Queens Problem
- Problem Statement
- Objectives
- Predicate Calculus
- Knowledge Representation

Part II – Implementation

- Logical Constraints
- Z3 Solver
- System Architecture
- AI and User Modes
- Testing and Validation

Project Overview

Part III – Application Features

- Board Sizes from 1 to 13
- Interactive Chessboard
- AI-Based Solution
- Hint Generation
- Timer and Win Detection
- Light and Dark Themes

Part IV – Evaluation

- Satisfiability Checking
- Solution Validation
- Unsatisfiable Cases
- Constraint-Based Reasoning
- Limitations
- Future Enhancements

Introduction

What is the N-Queens Problem?

The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other.

Core Constraints

- Exactly one queen in each row.
- No two queens share a column.
- No two queens share a diagonal.

AI Perspective

The problem can be represented as a set of logical constraints and solved by searching for an assignment that satisfies all constraints.

Problem Statement

Problem Definition

Given an integer N, determine a placement of N queens on an $N \times N$ chessboard such that every queen satisfies the non-attacking constraints.

Input:

- Board size N, where $1 \leq N \leq 13$.
- Optional fixed queen placements.

Output:

- A valid board configuration, if one exists.
- An unsatisfiable result if no valid configuration exists under the given constraints.

Important Observation

For the standard problem, solutions exist for $N = 1$ and all $N \geq 4$. No solution exists for $N = 2$ or $N = 3$.

Project Objectives

1. Represent the N-Queens problem using predicate-calculus concepts.
2. Express row, column, and diagonal requirements as logical constraints.
3. Encode the finite constraints using the Z3 solver.
4. Generate valid solutions through satisfiability checking.
5. Develop an interactive application for solving and playing the puzzle.
6. Validate the solver using satisfiable and unsatisfiable test cases.

Predicate Calculus Representation

Predicate Definition

Let $Q(r, c)$ denote:

$$Q(r,c) = \begin{cases} \text{True}, & \text{if a queen occupies } (r,c), \\ \text{False}, & \text{otherwise.} \end{cases}$$

The variables r and c represent row and column indices.

Finite Domain

$$r, c \in \{0, 1, \ldots, N-1\}$$

The board is represented as a finite domain over which the logical constraints are instantiated.

Goal: Find an assignment to the predicates that satisfies every constraint.

Logical Constraint I – Row Uniqueness

Every row must contain exactly one queen.

At Least One Queen in Every Row

$$\forall r \; \exists c \; Q(r, c)$$

This ensures that every row contains a queen.

At Most One Queen in Every Row

$$\forall r, c_1, c_2, \quad \big(Q(r, c_1) \land Q(r, c_2)\big) \Rightarrow c_1 = c_2$$

This prevents multiple queens from occupying the same row.

Combined Requirement

Every row contains exactly one queen.

Logical Constraint II – Column Uniqueness

No two queens may occupy the same column.

Predicate-Calculus Formula

$$\forall r_1, r_2, c, \quad \big(Q(r_1, c) \wedge Q(r_2, c)\big) \Rightarrow r_1 = r_2$$

Valid Placement

Queens occupy different columns.

Invalid Placement

Two queens in the same column violate the constraint.

Purpose: Eliminate board configurations containing a vertical attack.

Logical Constraint III – Diagonal Safety

Two queens attack each other diagonally when the absolute difference between their row indices equals the absolute difference between their column indices.

Predicate-Calculus Formula

$$\begin{aligned}\forall r_1, r_2, c_1, c_2, \quad & \big(Q(r_1, c_1) \land Q(r_2, c_2) \\ & \land r_1 \neq r_2\big) \\ & \Rightarrow |r_1 - r_2| \neq |c_1 - c_2|\end{aligned}$$

Interpretation

If two queens are in different rows, their row and column differences must not be equal in magnitude.

Combined Logical Model

The solution must satisfy all the following requirements.

1. **Row constraint:** Exactly one queen in every row.
2. **Column constraint:** No two queens in the same column.
3. **Diagonal constraint:** No two queens on the same diagonal.
4. **Fixed-position constraint:** Any queen specified by the user must remain in its designated square.

Satisfiability

Let C denote the conjunction of all constraints.

$$C = C_{\text{row}} \wedge C_{\text{column}} \wedge C_{\text{diagonal}} \wedge C_{\text{fixed}}$$

The solver searches for an assignment satisfying C.

Z3 Constraint Solver

What is Z3?

Z3 is an SMT (Satisfiability Modulo Theories) solver that determines whether a set of logical constraints can be satisfied and can produce a satisfying model.

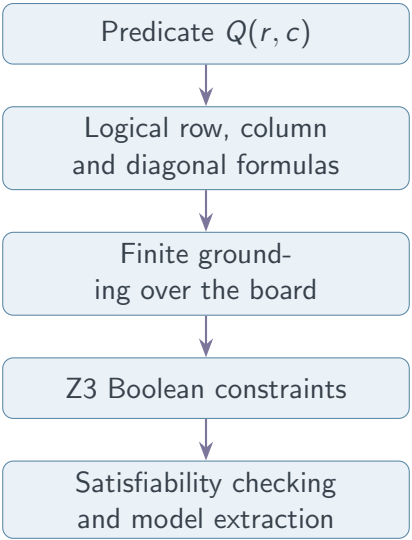
In This Project

- Boolean variables represent board squares.
- Row constraints use pseudo-Boolean equality.
- Column and diagonal constraints prohibit conflicting queen placements.
- Fixed queens become additional constraints.

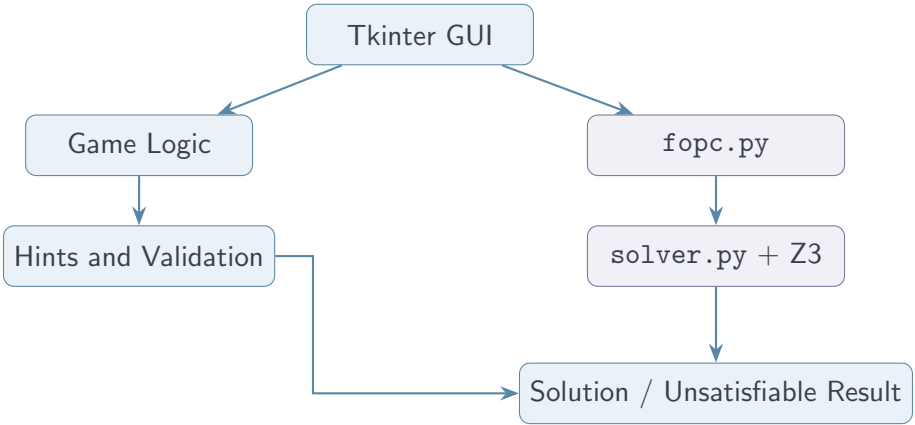
Solver Results

- `sat`: a valid assignment exists.
- `unsat`: constraints cannot be satisfied.
- A satisfying model identifies queen positions.

From Predicate Calculus to Z3



System Architecture



The GUI handles interaction, while the logical model and Z3 solver handle constraint-based reasoning.

Application Workflow

1. Select the board size.
2. Choose AI mode or user mode.
3. Generate logical constraints for the board.
4. Ask Z3 to find a satisfying assignment when an AI solution is requested.
5. Display the resulting queen positions.
6. In user mode, validate each move against the non-attacking rules.
7. Detect a completed valid board and display the win message.

Hint Mechanism

A hint can be generated by asking the solver for a valid completion consistent with the current queen placements.

Application Features

Solving Features

- Board size from 1 to 13.
- AI solution generation.
- User-controlled queen placement.
- Constraint-based move validation.
- Hint generation.

Interface Features

- Interactive chessboard.
- Light and dark themes.
- Game timer.
- Win detection and feedback.
- Fixed queen placements, where supported.

Testing and Validation

Board Size	Expected Result	Purpose
$N = 1$	SAT	Smallest valid board
$N = 2$	UNSAT	No valid arrangement
$N = 3$	UNSAT	No valid arrangement
$N = 4$	SAT	Valid solution exists

Additional Tests

- Verify that returned queen placements do not conflict.
- Supply conflicting fixed queen positions and expect an unsatisfiable result.
- Check board sizes at the supported boundaries.
- Test user moves, hints and win detection.

Efficiency and Optimization

Boolean Board Representation

The model creates a Boolean variable for every square.

$$\text{Number of Boolean variables} = N^2$$

For a board of size 13×13, this gives 169 Boolean variables.

Constraint-Based Solving

- Z3 searches for an assignment satisfying the logical constraints.
- Fixed queens restrict the search space.
- The solver can detect inconsistent constraints.
- The model avoids maintaining a separate backtracking solver.

The runtime depends on the solver, the constraints, and the instance. The number of variables alone does not establish a runtime guarantee.

Limitations and Scope

Current Scope

- The logical model is grounded over a finite chessboard.
- Z3 solves the resulting constraints; it is not a general-purpose first-order logic theorem prover.
- GUI behavior and solver behavior must be tested separately.
- Performance should be measured rather than assumed from the board-size limit.

Possible Enhancements

- Compare Z3 with a backtracking implementation.
- Record solving times across board sizes.
- Add step-by-step logical explanations.
- Improve hint explanations and move feedback.

Conclusion

- The N-Queens problem can be represented using predicates and logical constraints.
- Row, column and diagonal rules define valid board configurations.
- Finite grounding translates the logical model into constraints that Z3 can solve.
- Satisfiability checking provides a systematic method for finding solutions or detecting contradictions.
- The GUI combines logical reasoning with interactive gameplay.

Key Takeaway

The project demonstrates how logical knowledge representation can be connected to constraint-based reasoning in an AI application.

References I



L. de Moura and N. Bjørner,
"Z3: An Efficient SMT Solver,"
Tools and Algorithms for the Construction and Analysis of Systems (TACAS), 2008.



Microsoft Research,
Programming Z3.
Z3 Solver Documentation.
`https://microsoft.github.io/z3guide/`



S. Russell and P. Norvig,
Artificial Intelligence: A Modern Approach.
Pearson.

Thank You